

OpenCode Core 代码架构分析报告

一、项目概述

OpenCode 是一个开源的 AI 编程助手 (AI Coding Agent)，仓库地址 <https://github.com/anomalyco/opencode>，MIT 协议。

`@opencode-ai/core` (`packages/core`) 是整个项目的中枢核心包，负责所有底层能力：AI Provider 抽象、会话管理、插件系统、Git/文件系统操作、权限控制等。上层包 (CLI/TUI、桌面应用、Web 控制台) 都依赖它。

二、核心架构范式：Effect-TS 服务层模式

整个 core 采用 **Effect-TS v3** 构建。Effect-TS 是一个类型安全的函数式编程框架，提供：

原语	用途
<code>`Effect`</code>	带类型化成功(A)、错误(E)和依赖(R)的核心计算类型
<code>`Context.Service`</code>	依赖注入标签
<code>`Layer`</code>	依赖注入容器，组合式服务提供者
<code>`Schema`</code>	数据模式解析/验证 (替代 zod/io-ts)
<code>`ManagedRuntime`</code>	预组装的运行时，桥接 Effect ↔ 外部世界
<code>`Stream / PubSub`</code>	响应式流 / 发布订阅事件总线
<code>`Scope / Fiber`</code>	资源生命周期管理 / 并发绿色线程
<code>`SynchronizedRef`</code>	并发安全的可变状态

每个模块遵循统一配方：

- Branded ID 类型** → `Schema.String.pipe(Schema.brand("AgentID"))`
- Schema.Class/Struct** → 领域数据模型
- Interface** → 服务 API 契约
- Context.Service** → 依赖注入标签
- Layer.effect()** → 服务实现
- defaultLayer** → 完整依赖图 (Layer.provide 链)

没有 `index.ts`，通过 `package.json` 的 `"exports": { ".*": "./src/*.ts" }` 直接按文件路径导入。

三、关键模块详解

3.1 agent.ts — Agent 管理

管理 AI Agent 定义及其配置。支持 `subagent` / `primary` / `all` 三种模式。基于 `HashMap` 内存存储，`Immer` 不可变更新，`CRUD` 操作时触发插件钩子。

3.2 会话事件溯源管道

这是会话的**核心创新**——所有会话状态从事件流中派生：

```
session-event.ts      → 定义事件 (step/text/tool/reasoning/compaction)
    ↓
session-message-updater.ts → Reducer: 事件 → 消息状态转换 (纯函数)
    ↓
session-message.ts     → 最终视图 (User/Assistant/Shell/Synthetic 消息)
```

定义的事件包括：`step.started/ended/failed`、`text.started/delta/ended`、`tool.input.*/called/progress/success/failed`、`reasoning.*`、`compaction.*`、`shell.*` 等。

3.3 provider.ts — Provider 抽象

定义 LLM Provider 配置：端点类型（`openai/responses`、`openai/completions`、`ai-sdk`、`anthropic/messages`）、认证方式（`env/account/custom`）、请求选项（`headers`、`body`）。附带知名 Provider 的静态 ID。

3.4 model.ts — 模型定义

定义 LLM 模型元数据：能力（工具支持、MIME 类型）、分层定价（上下文窗口大小、输入/输出/缓存费率）、发布日期、状态（`alpha/beta/deprecated/active`）、限制（上下文/输入/输出 tokens）。

3.5 event.ts — 事件系统

基于 `Effect PubSub` 构建的类型化事件系统。支持按事件类型分别订阅、全局广播、同步处理器注册。用作整个项目的事件溯源基础层。

3.6 catalog.ts — Provider & 模型注册表

插件驱动的注册表。插件通过 `catalog.transform` 钩子启用 Provider、注入凭证。提供 `default` 和 `small` 模型选择算法（按成本/发布日期加权）。

3.7 aisdk.ts — AI SDK 桥接

OpenCode 模型定义与 `@ai-sdk/*` 生态系统之间的桥梁。通过插件钩子（`aisdk.sdk`、`aisdk.language`）创建 `LanguageModelV3` 实例。实现自定义 `fetch`（`SSE` 块超时、`AbortSignal` 组合）。

3.8 project.ts — 项目身份

通过分析 git 仓库解析项目 ID：读取缓存 → git 远程 URL 哈希 → 根提交哈希 → global 回退。

3.9 account.ts — 账户凭证

管理 OAuth 和 API Key 凭证。使用 SynchronizedRef 保证并发安全。在 CRUD 时发布事件通知。

四、插件系统（架构亮点）

以钩子驱动 + Effect 依赖注入 + Immer 结构共享为核心：

7 种标准钩子：

钩子	用途	触发者
`catalog.transform`	启用 Provider、注入凭证/headers	Catalog 重建时
`aisdk.sdk`	创建 AI SDK 实例	模型调用时
`aisdk.language`	选择语言模型	模型调用时
`account.switched`	账户切换通知	凭证变更时
`agent.update`	Agent 更新前拦截	Agent 服务
`agent.remove`	Agent 删除前拦截	Agent 服务
`agent.default`	覆盖默认 Agent 选择	Agent 服务

每个插件通过 PluginV2.define({ id, effect }) 定义，在独立 Scope 中运行。输出使用 Immer Draft 机制允许插件安全地原地修改，最后统一 finishDraft 得到最终结果。

插件注册顺序（有严格依赖）：

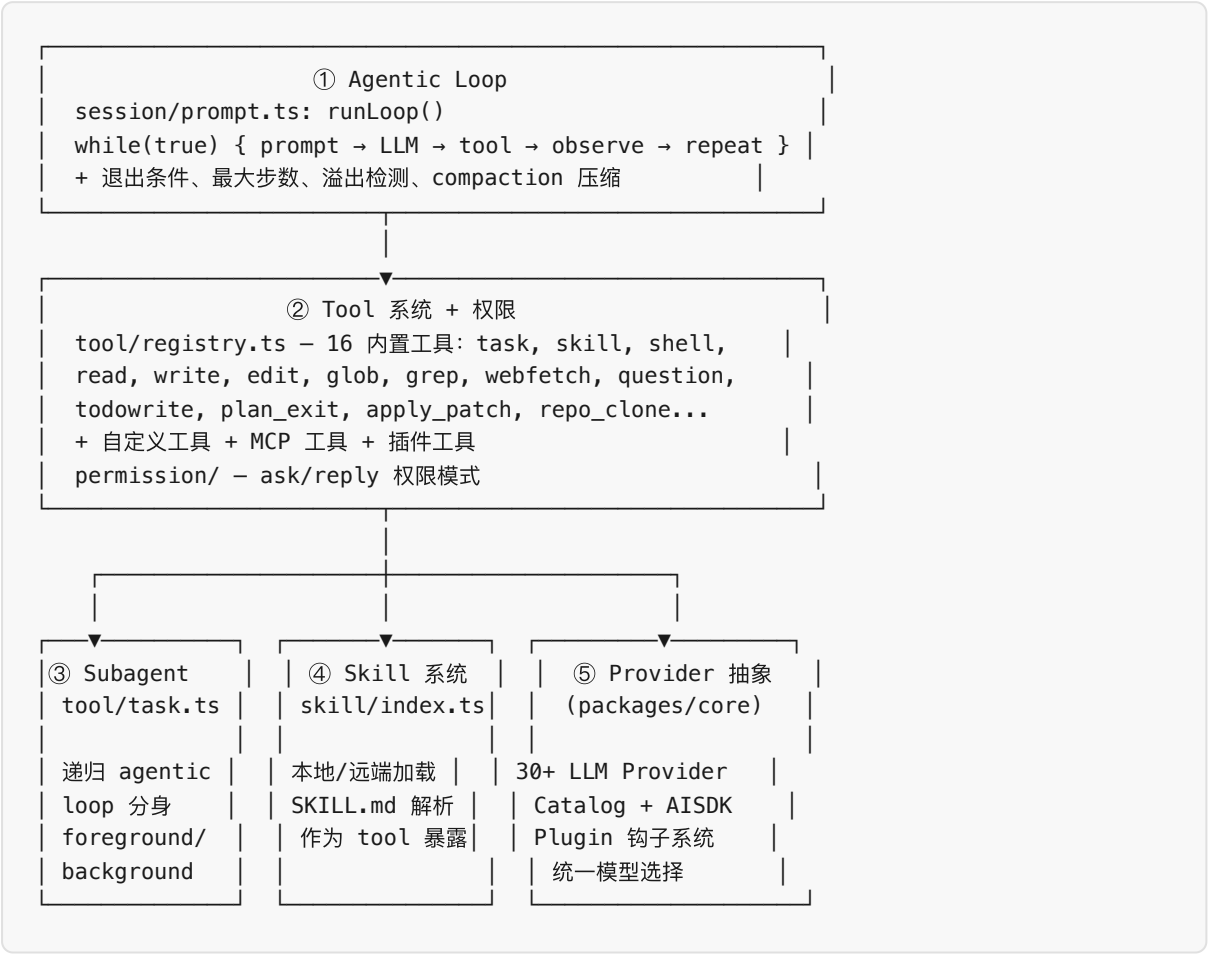
1. EnvPlugin — 检查环境变量，自动启用对应 Provider
2. AccountPlugin — 注入 API Key / OAuth Token
3. 30+ ProviderPlugins — OpenAI、Anthropic、Google、AWS Bedrock、Azure、GitHub Copilot 等
4. ModelsDevPlugin — 从外部源加载模型详细元数据

Provider 插件类型：

- 极简模式（Anthropic、OpenAI 等）：注入 headers + 动态 import AI SDK
- 复杂模式（Amazon Bedrock）：动态凭证链、跨区域推理
- 特殊模式（Google Vertex）：认证 fetch 注入、URL 模板替换
- 动态模式（DynamicProviderPlugin）：运行时 npm 安装未知 Provider，自动发现工厂函数

五、完整的 AI Agent Harness 五层架构

core 包只覆盖 Provider 抽象层。完整的 Harness 还包括 `packages/opencode` 中的执行引擎：



Subagent 机制： `task` tool 让 LLM 分身出子 agent，每个子 agent 拥有独立 session、独立 agentic loop、被限制的权限集（不能递归再调 `task`，防止无限分叉）。支持 `foreground`（阻塞等待）和 `background`（异步执行）两种模式。

Skill 系统： 从 `~/.claude/skills/`、项目 `.opencode/skills/`、远端 URL 等加载 `SKILL.md`，解析 frontmatter 元数据，作为 tool 暴露给 LLM。LLM 调用后返回技能指令文本，注入到后续 prompt 中改变 agent 行为。

六、工具库（`src/util/`）

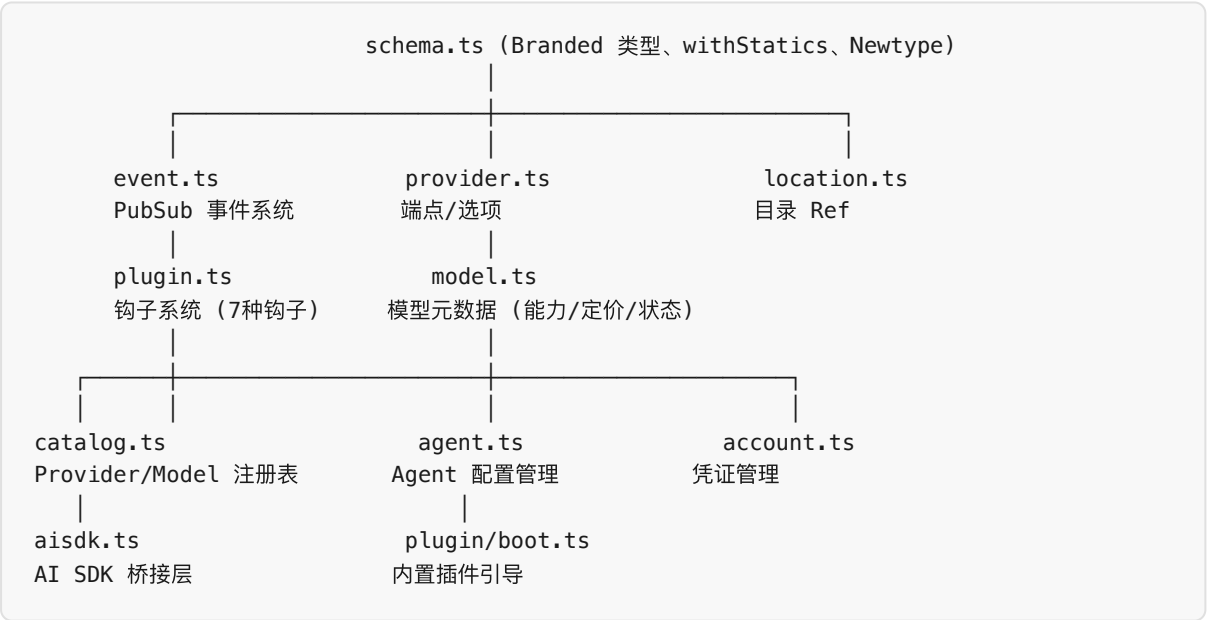
类别	文件
并发控制	`flock.ts`（Promise版）、`effect-flock.ts`（Effect版）— 文件锁，过期检测/心跳
标识符	`identifier.ts`（可排序 26 字符 ID）、`slug.ts`（adjective-noun）、`hash.ts`（SHA1）

编码加密	<code>`encode.ts`</code> (Base64 / SHA-256 / FNV-1a 采样校验和)
错误处理	<code>`error.ts`</code> (带 Schema 的结构化 NamedError)
日志系统	<code>`log.ts`</code> (DEBUG/INFO/WARN/ERROR, 标签系统, 自动清理旧日志)
重试机制	<code>`retry.ts`</code> (指数退避, 临时性网络错误自动重试)
文件操作	<code>`glob.ts`</code> (通配符扫描)、 <code>`wildcard.ts`</code> (轻量匹配)、 <code>`path.ts`</code> (跨平台路径)

七、Effect 基础设施 (``src/effect/``)

- **runtime.ts** — `makeRuntime(Service, layer)` 桥接 Effect ↔ async/await, 延迟构建 ManagedRuntime, 自动注入可观测性层
- **logger.ts** — 自定义日志器, 桥接 Effect Logger → Log 服务
- **observability.ts** — 条件性 OpenTelemetry 集成 (OTLP 端点 → 日志+追踪)
- **service-use.ts** — 类型安全的惰性服务访问器, 消除 `Service.use()` 样板代码

八、核心依赖图



基础设施层： `global.ts` (XDG 目录)、`filesystem.ts` (增强文件系统)、`git.ts` (仓库操作)、`process.ts` (子进程管理)、`flag/flag.ts` (功能开关)

九、学习路线建议

1. **schema.ts** — 理解 Branded ID、withStatics、Newtype 等基础
 2. **event.ts** — 理解 PubSub 事件系统
 3. **provider.ts + model.ts** — 理解 AI 模型抽象
 4. **plugin.ts + plugin/boot.ts** — 理解插件/钩子系统
 5. **catalog.ts + aisdk.ts** — 理解 Provider 发现和 AI SDK 桥接
 6. **session-event.ts + session-message-updater.ts** — 理解事件溯源
 7. **effect/runtime.ts** — 理解 Effect ↔ 外部世界桥接
 8. **packages/opencode/src/session/prompt.ts** — 理解 agentic loop 主循环
 9. **packages/opencode/src/tool/registry.ts + task.ts** — 理解工具/Subagent 系统
 10. **packages/opencode/src/skill/index.ts** — 理解 Skill 系统
-

本报告基于 OpenCode v1.15.11 源码分析生成，所有路径相对于项目根目录。